



Étude de qualité de données d'une base de données graphe

TER M1 Informatique

Étude de qualité de données d'une base de données graphe

Auteur : Loïc DESMARÈS et Tianyi YANG

Encadrantes : Béatrice FINANCE et Zoubida KEDAD

Université Paris-Saclay – UVSQ

Année universitaire 2025–2026

Table des matières

1 - Introduction	3
2 - Qualité de données d'un Graphe de Propriété	4
2.1 - Complétude	4
2.1.1 - Existence de composantes	4
2.1.2 - Le degré des noeuds	4
2.2 - Conformité	4
2.2.1 - Format des chaînes de caractères	4
2.2.2 - Format des dates	4
2.2.3 - Ensemble fini de données	4
2.2.4 - Étiquetage Ensembliste	5
2.2.5 - Étiquetage par Regroupement (clustering)	5
2.3 - Cohérence	8
2.3.1 - Dépendance Fonctionnelle (FD)	8
2.3.2 - Dépendance Fonctionnelle Conditionnelle (CFD)	8
2.3.3 - Dépendance d'un Graph pattern (GFD)	8
2.3.4 - Validation par requête	8
2.4 - Intégrité	9
2.4.1 - Validité du schéma de propriété	9
2.4.2 - Validité des Index	9
2.4.3 - Forme normale d'un Graphe de propriété	9
2.5 - Unicité	10
2.5.1 - Doublons d'arcs	10
2.5.2 - Doublons de noeuds	10
3 - Profilage d'un Graphe de Propriété	11
3.1 - Complétude	11
3.1.1 - Composants faiblement connectés	11
3.1.2 - Composants fortement connectés	11
3.2 - Conformité	11
3.2.1 - Détection de types distincts pour des propriétés	11
3.3 - Intégrité	11
3.3.1 - Distribution des propriétés des noeuds	11
3.3.2 - Distribution des propriétés des noeuds par étiquette	11
3.3.3 - Distribution des propriétés des arcs	11
3.4 - Étiquetage	11
3.4.1 - Détection d'anomalies par regroupement (clustering)	11
3.5 - Lisibilité	12
3.5.1 - Distribution du degré des noeuds	12
3.5.2 - Détection des arcs formant un multigraphe	12
3.5.3 - Analyse de l'excentricité du graphe	12
3.6 - Valeurs aberrantes (<i>Outlier</i>)	12
3.6.1 - Détection des valeurs numériques aberrantes	12
3.6.2 - Analyse de l'influence transitive des noeuds	12
3.6.3 - Analyse de l'influence transitive moyenne	12
4 - Implémentation - Neo4j	13
4.1 - Méthodes de test	13
4.2 - Résultats des tests	13
4.2.1 - Northwind	13
4.2.2 - YAGO3	14
5 - Conclusion	14
6 - Annexe	15
Bibliographie	30

Résumé : L’essor du numérique et la multiplication des sources de données hétérogènes ont conduit à une adoption croissante des bases de données orientées graphes, et plus particulièrement du modèle de graphe de propriétés (Property Graph). Ce modèle se distingue par sa richesse sémantique et sa flexibilité structurelle : en permettant d’associer à chaque noeud et à chaque arc un ensemble d’étiquettes et de propriétés. Cela offre une représentation naturelle et expressive des entités et de leurs relations au sein de domaines variés tels que les réseaux sociaux, la bioinformatique, la gestion de connaissances à grande échelle ou encore les systèmes d’intelligence artificielle.

Bien que l’état de l’art et la standardisation des graphes de propriété a beaucoup évolué, les aspects de qualité de données demeurent largement inexplorés. Or, à l’instar des environnements de données massives (*Big Data*), la valeur extraite d’une base de données graphe repose fondamentalement sur la fiabilité, la cohérence, la complétude et l’exactitude des données qu’elle contient. La présente étude propose d’analyser et d’adapter les cadres existants d’évaluation de la qualité des données au contexte spécifique des graphes de propriétés, en tenant compte des contraintes propres à ce modèle — notamment l’absence de schéma rigide, la multiplicité des sources et la diversité des types de données. Nous formulons un ensemble de dimensions et d’indicateurs de qualité pertinents pour ce contexte, et discutons des méthodes d’évaluation envisageables, posant ainsi les bases d’un processus d’évaluation de la qualité des données adapté aux bases de données graphes.

1 - Introduction

Cette étude a pour objectif de déterminer des critères de qualité de données pour les bases de données graphe. On considérera ici les **Graphes de Propriété** (*Property Graph*) disposant d’**étiquettes** (*labels*).

Définition 1.

Un graphe de propriété est un tuple $G = (N, E, \rho, \lambda, \sigma)$ tel que :

1. N est un ensemble fini de noeuds (*nodes*), aussi appelé sommets (*vertices*).
2. E est un ensemble fini d’arcs (on parlera d’arête lorsque la direction n’est pas prise en compte).
3. $\rho : E \rightarrow (N \times N)$ est une fonction totale qui associe pour chaque arc dans E une paire $(n_{\text{source}}, n_{\text{destination}})$. Cette paire de noeuds est donc non commutative car $(n_A, n_B) \in \rho(E)$ n’implique pas nécessairement $(n_B, n_A) \in \rho(E)$.
4. $\lambda : (N \cup E) \rightarrow \text{SET}^+(L)$ est une fonction partielle qui associe à un noeud ou un arc un ensemble d’étiquettes incluses dans L (λ est une fonction d’étiquetage des noeuds et des arcs).
5. $\sigma : (N \cup E) \times P \rightarrow \text{SET}^+(V)$ est une fonction partielle qui associe aux noeuds et arcs des valeurs V aux propriétés P . On notera par la suite, $\forall (o, p) \in (N \cup E) \times P$ et ses valeurs assignées $\sigma(o, p) = \{v_1, \dots, v_n\}$ par $(o, p) = \vec{v}$.

2 - Qualité de données d'un Graphe de Propriété

2.1 - Complétude

Définition 2.1.0

La Complétude mesure la quantité de données manquantes d'une base de données graphe [1].

2.1.1 - Existence de composantes

L'existence de composantes connexes ou fortement connexes est une méthode pour vérifier la complétude des données. En effet, les arcs modélisent une grande partie des relations entre les objets et sont porteurs d'un sens sémantique important.

Plus intuitivement vérifier l'existence de composantes entre des ensembles de noeuds et d'arcs permet par exemple d'exprimer des contraintes de chemins (resp. chaînes). Naturellement l'ajout de contraintes comme la longueur des chemins, l'appartenance d'un ensemble d'étiquettes à ceux-ci constituent de solides outils pour capturer un sens sémantique complexe.

Définition 2.1.1

Soit G_p un graph pattern (*patron de graphe*) modélisant une composante connexe (resp. fortement connexe), un ensemble O tel que $O \in \{N, E, N \cup E\}$ et $L_O \subseteq L$ est un ensemble d'étiquettes. Tel que $\forall (G_p, O, L_O)$ on a $\forall o \in O$ tel que $\lambda(o) = L_O$, $\exists$ au moins une occurrence de G_p pour o . Cf. **Figure 1** en annexe.

2.1.2 - Le degré des noeuds

La complétude des données peut aussi s'exprimer par le degré des noeuds et ainsi exprimer des contraintes de cardinalités.

Définition 2.1.2

Soit $L_O \subseteq L$ est un ensemble d'étiquettes et les ensembles $D_s, D_e \subset \mathbb{R}^2$, représentant respectivement l'ensemble des degrés sortant et entrant. Tel que $\forall n \in N$ tel que $\lambda(n) = L_O$ vérifie $d^+(n) \in D_s$ et $d^-(n) \in D_e$. Cf. **Figure 2** en annexe.

2.2 - Conformité

Définition 2.2.0

La Conformité mesure la validité du format des données.

2.2.1 - Format des chaînes de caractères

Définition 2.2.1

Soit $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$ et **Regex** codant le format attendu. Tel que $\forall o \in O$ on vérifie que $\forall v \in \sigma(o, X)$, $\text{match}(v, \text{Regex}) = \text{Vrai}$. Cf. **Figure 3** en annexe.

2.2.2 - Format des dates

Définition 2.2.2

Soit $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$ et **Date_{fnt}** codant le format de date attendu. Tel que $\forall o \in O$ on vérifie que $\forall v \in \sigma(o, X)$, $\text{match}(v, \text{Date}_{\text{fnt}}) = \text{Vrai}$. Cf. **Figure 4** en annexe.

2.2.3 - Ensemble fini de données

Définition 2.2.3

Soit $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$, I un ensemble de données (non atomique comprises) et C une contrainte optionnelle (cf. Définition 2.3.2). Tel que $\forall o \in O$ on vérifie $\text{SET}(\sigma(o, X)) \subseteq I \wedge C(o) = \text{Vrai}$. Cf. **Figure 5** en annexe.

2.2.4 - Étiquetage Ensembliste

Définition 2.2.4

Soit $O \in \{N, E, N \cup E\}$, $L_X, L_Y \subseteq L$ et $\text{Op}_{\text{ens}} \in \{\subset, \subseteq, \setminus\}$ un opérateur ensembliste. Tel que $\forall o \in O$ tel que $L_X \subseteq \lambda(o)$ vérifie $\lambda(o)\text{Op}_{\text{ens}}L_Y = \text{Vrai}$. Cf. **Figure 6** en annexe.

Notons que seule l'implémentation partielle de cette définition à du sens dans le cadre des bases de données graphe **Neo4j**, car les arcs (*Relationships*) ne peuvent avoir qu'une seule étiquette.

2.2.5 - Étiquetage par Regroupement (clustering)

L'intuition est la suivante : des noeuds similaires doivent avoir le même ensemble d'étiquettes. Pour mesurer la qualité de l'étiquetage, on cherche donc à regrouper les noeuds similaires pour détecter les erreurs d'étiquetage. L'approche qui suit est inspirée d'un système d'embeddings motivé par l'article [2]. L'approche proposée est la suivante :

1. Déterminer un critère de similarité entre deux noeuds : on s'intéresse ici aux étiquettes des noeuds donc au sens sémantique de celles-ci. Notre intérêt se porte donc sur les relations entre les différents ensembles d'étiquettes. Ces relations sont ici modélisées par un concept riche en sémantique : les arcs. En effet les arcs sont caractérisés par une paire de noeuds (disposant d'une direction) et un ensemble d'étiquettes. On propose donc de traduire ce sens sémantique par des chaînes de caractères. Ainsi l'arc suivant :

(Noeud₁: {Étudiant, Personne})-[Arc:{Inscrit}]->(Noeud₂: {Université})

Serait traduit par « OU:Inscrit:Université » (que l'on nomme un *Token*) du point de vue de Noeud₁ et par « IN:Inscrit:ÉtudiantPersonne » de celui de Noeud₂.

2. Déterminer une méthode de calcul de similarité entre deux *Token*. Sachant qu'un *Token* traduit des relations sémantiques complexes par une chaîne de caractères, l'utilisation de la distance d'édition (*Edit distance*) semble le plus adapté. On utilise donc la similarité de **Levenshtein** pour calculer la similarité entre deux *Token*.
3. Déterminer une méthode de calcul de similarité entre deux noeuds. On s'intéresse à leurs relations et à leurs étiquettes, on va donc combiner un score de similarité de ces deux dimensions. On utilise l'indice de **Jaccard** pour calculer la similarité entre deux noeuds sur le critère des ensembles d'étiquettes, tel qu'on a $\forall n_1, n_2 \in N^2$, $\text{Similarité}_{\text{Étiquettes}} = 1 - \frac{|\lambda(n_1) \cap \lambda(n_2)|}{|\lambda(n_1) \cup \lambda(n_2)|}$.

La similarité entre deux noeuds sur le critère des *Tokens* est calculée avec la similarité **Monge-Elkan** (ME), tel qu'on a $\forall n_1, n_2 \in N^2$, $\text{ME}(n_1, n_2) = \frac{1}{|\sigma(n_1, \{\text{Tokens}\})|} \sum_{t_1 \in \sigma(n_1, \{\text{Tokens}\})} \max_{t_2 \in \sigma(n_2, \{\text{Tokens}\})} (\text{similarité_levenshtein}(t_1, t_2))$ (resp. $\text{ME}(n_2, n_1)$), tel que $\text{Similarité}_{\text{Tokens}} = \frac{\text{ME}(n_1, n_2) + \text{ME}(n_2, n_1)}{2}$. La similarité des *Tokens* pourrait aussi être calculée à l'aide de l'algorithme **Fuzzy Jaccard** (moins précis) ou encore de l'algorithme de **Kuhn-Munkres** (optimal mais en $O(|N|^3)$).

On définit donc les algorithmes suivants :

L'algorithme *Tokenization* permet de générer un ensemble de *Token* (représentant les relations d'un noeud) pour tous les noeuds tels qu'il existe un arc entrant ou sortant de ceux-ci. Autrement formulé : tout noeud ayant un degré entrant ou sortant non nul dispose d'une propriété « Tokens » sauvegardant l'ensemble de des *Token* générés le concernant.

L'algorithme *CreateTokens* quant à lui génère un ensemble de noeuds connexes représentant l'ensemble des *Token* distincts générés par l'algorithme *Tokenization*. Une fois ces noeuds créés,

l'algorithme *CreateTokens* calcule la similarité de **Levenshtein** entre chaque paire de *Token* et la sauvegarde sous forme d'arc entre ceux-ci.

Tokenization(*G* PG.):

```

1 // Étape - 1
2 for e in E do
3   (ns, nd) ←  $\rho(e)$ 
4   cs ← "OU:" +  $\lambda(e)$  + ":" +  $\lambda(n_d)$ 
5   cd ← "IN:" +  $\lambda(e)$  + ":" +  $\lambda(n_s)$ 
6   ts ←  $\sigma(n_s, \{\text{Tokens}\})$ 
7   td ←  $\sigma(n_d, \{\text{Tokens}\})$ 
8    $\sigma(n_s, \{\text{Tokens}\}) \leftarrow t_s \cup c_s$ 
9    $\sigma(n_d, \{\text{Tokens}\}) \leftarrow t_d \cup c_d$ 
10 end

```

L'algorithme *Tokenization* a une complexité temporelle linéaire en $O(n)$ (avec $n = |E|$) et une complexité spatiale dans le pire cas (très rare) en $O(n \times m)$ (avec $n = |N|$ et $m = |L|$). L'algorithme *CreateTokens* a une complexité temporelle polynomiale en $O(n^2)$ (avec $n = |N|$) et une complexité spatiale dans le pire cas (très rare) en $O(n^2)$ (avec $n = 2|E|$).

CreateTokens(*G* PG.):

```

1 // Étape - 2
2 vocab ←  $\emptyset$ 
3 for n in N do
4   tokens ←  $\sigma(n, \{\text{Tokens}\})$ 
5   if tokens ≠ NULL do
6     vocab ← vocab  $\cup$  tokens
7   for idx in [0; |tokens| - 1] do
8     n ← newNode()
9     N ← N  $\cup$  {n}
10     $\lambda(n) \leftarrow \{\text{TOKEN}\}$ 
11     $\sigma(n, \{\text{VAL}, \text{ID}\}) \leftarrow (\text{vocab}[\text{idx}], \text{idx})$ 
12  // On calcule la similarité entre les tokens.
13  for (n1, n2) in N2 do
14    if  $\lambda(n_1) = \lambda(n_2) = \{\text{TOKEN}\}$ 
15    and  $\sigma(n_1, \{\text{ID}\}) < \sigma(n_2, \{\text{ID}\})$  do
16      t1, t2 ←  $\sigma(n_1, \{\text{VAL}\}), \sigma(n_2, \{\text{VAL}\})$ 
17      sim ← Similarité_Levenshtein(t1, t2)
18      e ← newEdge()
19      E ← E  $\cup$  {e}
20       $\rho(e) \leftarrow (n_1, n_2)$ 
21       $\lambda(e) \leftarrow \{\text{SIMILAR}\}$ 
22       $\sigma(e, \{\text{SCORE}\}) \leftarrow \text{sim}$ 
23 end

```

Une fois les algorithmes *Tokenization* et *CreateTokens* on peut analyser les regroupements de noeuds avec leur similarité entre ensemble de *Token*, et sur la similarité de **Jaccard** pour calculer la similarité entre leurs étiquettes. Le regroupement s'effectue par paire de noeuds et on ne sauvegarde qu'un simple arc liant les noeuds qui devraient être « Merge » (ceux-ci devraient avoir un ensemble similaire d'étiquettes) ou « Split » (ceux-ci ne devraient pas avoir un ensemble similaire d'étiquettes). Cette sélection est déterminée avec deux seuils de similarité, le premier concerne la similarité entre les étiquettes (cela permet de filtrer les paires de noeuds qui pourraient présenter des erreurs d'étiquetage). Ainsi qu'un deuxième seuil concernant la similarité des *Tokens*, déterminant la création (ou non) d'un arc « Merge » / « Split ».

```

Merge( $G$  PG.,  $t_e$  seuil similarité étiquettes,  $t_t$  seuil similarité Tokens):
1 // Étape - 3 : Détection de noeuds qui devraient appartenir
2 // au même cluster d'étiquettes.
3 for ( $n_1, n_2$ ) in  $N^2$  do
4   if  $\sigma(n_1, \{ID\}) < \sigma(n_2, \{ID\})$ 
5   and  $\sigma(n_1, \{Tokens\}) \neq \text{NULL}$ 
6   and  $\sigma(n_2, \{Tokens\}) \neq \text{NULL}$  do
7     SimilaritéÉtiquettes  $\leftarrow \frac{|\lambda(n_1) \cap \lambda(n_2)|}{|\lambda(n_1) \cup \lambda(n_2)|}$ 
8     if SimilaritéÉtiquettes  $\geq t_e$  do
9       skip this iteration;
10    end
11
12    SimilaritéTokens  $\leftarrow \frac{\text{ME}(n_1, n_2), \text{ME}(n_2, n_1)}{2}$ 
13    if SimilaritéTokens  $\geq t_t$  do
14       $e \leftarrow \text{newEdge}()$ 
15       $E \leftarrow E \cup \{e\}$ 
16       $\rho(e) \leftarrow (n_1, n_2)$ 
17       $\lambda(e) \leftarrow \{\text{MERGE}\}$ 
18    end
19  end
20 end

```

L'algorithme *Merge* détaillé ci-dessus permet donc de détecter toutes les paires de noeuds dont la similarité des étiquettes est $<$ au seuil t_e ; et pour lesquelles la similarité des *Token* est $\geq$ au seuil t_t . En d'autres termes l'algorithme détecte les noeuds qui de par leur similarité de relations (*Token*), devraient avoir un ensemble d'étiquettes plus similaire (donc ils devraient être rassemblés). Cf. **Figure 7** en annexe.

Cet algorithme peut être aisément adapté pour détecter l'inverse : « Split » désignant les paires de noeuds qui ne devraient pas avoir des ensembles d'étiquettes similaires. Pour opérer les changements nécessaires il suffirait de modifier comme suit les lignes [8, 21, 25] de l'algorithme *Merge*.

```

Split( $G$  PG.,  $t_e, t_t$ ):
1 // [...]
2 // Ligne 8 :
3 if SimilaritéÉtiquettes  $\leq t_e$  do
4 // [...]
5 // Ligne 21 :
6 if SimilaritéTokens  $\leq t_t$ 
7   // [...]
8   // Ligne 25 :
9    $\lambda(e) \leftarrow \{\text{SPLIT}\}$ 

```

2.3 - Cohérence

Définition 2.3.0

La Cohérence mesure la validité des relations de la base de données graphe.

2.3.1 - Dépendance Fonctionnelle (FD)

Définition 2.3.1

Soit $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$ et $X, Y \subseteq P$, on définit par $(O, L_O, X \rightarrow Y)$ une **FD**. Tel que $\forall o_1, o_2 \in O^2$ tel que $\lambda(o_1) = L_O$ et $\lambda(o_2) = L_O$ vérifie $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow \sigma(o_1, Y) = \sigma(o_2, Y)$. Cf. **Figure 8** en annexe.

2.3.2 - Dépendance Fonctionnelle Conditionnelle (CFD)

Définition 2.3.2.a

Une **condition** est un tuple $C = (P_C, \text{VAL}, f, \text{NEXT})$ tel que :

1. $P_C \subseteq P$ est l'ensemble des propriétés devant respecter la condition.
2. $\text{VAL} \in \{\text{constante}, P\}$ est la valeur de comparaison. La « constante » peut être tout type de données (non atomique comprises).
3. $f : (N \cup E, P_C, \text{VAL}) \rightarrow \text{Booléen}$, est une fonction permettant de vérifier la condition sur un objet (ex. « = », « < », « ∈ », etc.). On notera par la suite $C(o)$ le fait que l'objet o vérifie $f(o, P_C, \text{VAL})$ sachant P_C et VAL définis dans C .
4. $\text{NEXT} \in \{\emptyset, (\text{Condition}, \text{Opérateur booléen})\}$ est une deuxième condition (optionnelle) devant être vérifiée (permettant ainsi de la combiner avec la première avec l' « Opérateur booléen »).

Définition 2.3.2.b

Soit $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, C une condition (cf. Définition 2.3.2) et $X, Y \subseteq P$, on définit par $(O, L_O, C, X \rightarrow Y)$ une **CFD**. Tel que $\forall o_1, o_2 \in O^2$ tel que $\lambda(o_1) = L_O$ et $\lambda(o_2) = L_O$ et $C(o_1) = \text{Vrai}$ et $C(o_2) = \text{Vrai}$ vérifie $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow \sigma(o_1, Y) = \sigma(o_2, Y)$. Cf. **Figure 9** en annexe.

2.3.3 - Dépendance d'un Graph pattern (GFD)

Définition 2.3.3

Soit G_p un graph pattern à partir duquel on déduit $G'(N', E')$, sous graphe de G correspondant à G_p ; $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$ et $X, Y \subseteq P$, on définit par $(O, L_O, G_p, X \rightarrow Y)$ une **GFD**. Tel que $\forall o_1, o_2 \in O^2$ tel que $o_1, o_2 \in G'$ vérifie $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow \sigma(o_1, Y) = \sigma(o_2, Y)$. Cf. **Figure 10** en annexe.

Une autre approche (très différente) nommée *gFD* [3] consiste à exclure tous les noeuds qui n'ont pas toutes les propriétés $(X \cup Y)$ définies par la *FD*.

2.3.4 - Validation par requête

Les dépendances fonctionnelles (*FD*, *CFD* et *GFD*) sont des outils très puissants. Mais ils sont complexes à étendre pour parvenir à capturer l'ensemble du sens sémantique offert par les requêtes. C'est pourquoi une approche de validation supplémentaire consisterait — sur le modèle de *dbt* — à valider ou invalider des requêtes écrites par l'utilisateur.

Définition 2.3.4

Ce système de validation par requête est défini par un tuple $dgt = (R, B)$ tel que :

1. R est une requête, aussi riche que le langage de requêtage le permet; qui renvoie (ou non) des objets.
2. B est un booléen indiquant si R doit renvoyer des objets pour valider la contrainte définie par celle-ci.

Cf. **Figure 11** en annexe.

2.4 - Intégrité

Définition 2.4.0

L'intégrité mesure la validité structurelle d'une base de données graphe.

2.4.1 - Validité du schéma de propriété

Dans l'état de l'art aucun standard *DDL* n'a émergé pour les bases de données graphe. On va donc définir trois contraintes d'intégrité :

1. Unicité de propriétés :

Soit $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$ et $X \subseteq P$ tel que $\forall o_1, o_2 \in O^2$ vérifie $\sigma(o_1, X) = \sigma(o_2, X) \Rightarrow o_1 = o_2$. Cf. **Figure 12** en annexe.

2. Existence de propriétés :

Soit $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$ et $X \subseteq P$ tel que $\forall o \in O$ vérifie $\text{NULL} \notin \sigma(o, X)$. Cf. **Figure 13** en annexe.

3. Type des valeurs de propriétés :

Soit $t : (V) \rightarrow \text{SET}^+(T)$ une fonction totale qui attribut un ensemble de type T à un ensemble de valeurs V , $O \in \{N, E, N \cup E\}$, $L_O \subseteq L$, $X \subseteq P$ et $Y \subseteq T$ tel que $\forall o \in O$ vérifie $(t \circ \sigma)(o, X) \subseteq Y$. Cf. **Figure 14** en annexe.

Notons que ces contraintes peuvent être définies en **Cypher** (le langage de requêtes de **Neo4j**).

2.4.2 - Validité des Index

L'intuition est la suivante : des valeurs manquantes sur des propriétés indexées peuvent être un signal de dégradation de l'intégrité de la base de données graphe.

Définition 2.4.2

Soit $i : (N \times E) \rightarrow \text{BITSET}$ des propriétés indexées, $\forall o \in (N \cup E)$ on vérifie $\text{NULL} \notin i(o) \odot \sigma(o, P)$. Cf. **Figure 15** en annexe.

2.4.3 - Forme normale d'un Graphe de propriété

Algorithme :

```
PG_3FN( $G$  PG.,  $F$  FD set,  $L$  Label set):
1  $F_{\min} \leftarrow \text{get\_min\_cover}(F)$ 
2  $D \leftarrow \emptyset$ 
3  $\forall (X \rightarrow A) \text{ in } F_{\min}$ :
4    $F_x \leftarrow F_{\min} - \{(X \rightarrow A)\}$ 
5   if  $(F_x \models X \rightarrow A)$ :
6      $F_{\min} \leftarrow F_x$ 
7   else
8      $D \leftarrow D \cup \{(XA, F_{\min}.\text{projection}(XA))\}$ 
9  $F^+ \leftarrow \text{atomic\_closure}(F)$ 
10 'LoopA  $\forall R_p$  in  $F^+$ :
11    $\forall (R, F_r)$ 
12   if  $(R \rightarrow RP) \in F^+$ :
13     continue 'LoopA
14 key  $\leftarrow \text{find\_key}(R_p, F^+)$ 
15  $D \leftarrow D \cup \{(\text{key}, F_{\min}.\text{projection}(\text{key}))\}$ 
```

```
get_min_cover( $F$  FD set):
1  $F_{\min} \leftarrow \text{min\_cover}(F)$ 
2  $\forall (X \rightarrow A) \text{ in } F_{\min}$ :
3    $F_x \leftarrow F_{\min} - \{(X \rightarrow A)\}$ 
4    $\forall (X \rightarrow B) \text{ in } F_x$ :
5     if  $(YB \subseteq XA)$ 
6     and  $(XA \not\subseteq Y^+)$ 
7     and  $(F_x \models X \rightarrow A)$ :
8      $F_{\min} \leftarrow F_x$ 
9 return  $F_{\min}$ 
```

L'algorithme est défini dans le cadre des **gFD** et **gUC** [4] que l'on peut facilement traduire par les **FD** (cf. Définition 2.3.1). Tandis que les **CFD** et les **GFD** (graph pattern FD), n'ont

pas de sens dans un contexte de normalisation car l'algorithme normaliserait en 3NF seulement un fragment de la base de données. Cf. **Figure 16** en annexe.

2.5 - Unicité

Définition 2.5.0

L'unicité mesure la redondance d'une base de données graphe.

2.5.1 - Doublons d'arcs

Définition 2.5.1

$\forall e_1, e_2 \in E^2$, e_1 et e_2 sont des doublons si et seulement si : $\rho(e_1) = \rho(e_2)$, $\lambda(e_1) = \lambda(e_2)$ et $\sigma(e_1, P) = \sigma(e_2, P)$. Cf. **Figure 17** en annexe.

2.5.2 - Doublons de noeuds

Définition 2.5.2

$\forall n_1, n_2 \in N^2$, n_1 et n_2 sont des doublons si et seulement si : $\lambda(n_1) = \lambda(n_2)$ et $\sigma(n_1, P) = \sigma(n_2, P)$. Cf. **Figure 18** en annexe.

Cette définition pourrait être assouplie en prenant aussi en compte les arcs des noeuds et ainsi stipuler qu'au-dessus d'un certain seuil d'arcs en commun, ceux-ci sont considérés comme des doublons.

3 - Profilage d'un Graphe de Propriété

L'objectif du profilage d'une base de données graphe est d'avoir un tableau de bord sur la distribution des données. Cette section regroupe donc des indicateurs intéressants pour caractériser les données d'un graphe de propriété. Ces indicateurs ne constituent pas des éléments de qualité de données car la nature généraliste de ceux-ci ne saurait capturer les usages métier intrinsèques à une base de données graphe.

3.1 - Complétude

3.1.1 - Composants faiblement connectés

Définition 3.1.1

Détection des composantes connexes du graphe avec l'algorithme **WCC**.

3.1.2 - Composants fortement connectés

Définition 3.1.2

Détection des composantes fortement connexes du graphe avec l'algorithme **SCC** (on ne considère ici que les chemins).

3.2 - Conformité

3.2.1 - Détection de types distincts pour des propriétés

Définition 3.2.1

$\forall p \in P$ on vérifie que $\forall o \in N$ (resp. E), tel que $\sigma(o, \{p\}) \neq \text{NULL}$, $\exists$ un unique type t_x tel que $(t \circ \sigma)(o, \{p\}) = t_x$ (cf. Définition 2.4.1). Un tableau de bord concis listant les propriétés pour lesquelles le type n'est pas unique est construit à partir de cette détection.

3.3 - Intégrité

3.3.1 - Distribution des propriétés des noeuds

Définition 3.3.1

Analyse de la distribution des propriétés définies pour des noeuds, regroupés selon leur ensemble d'étiquettes.

3.3.2 - Distribution des propriétés des noeuds par étiquette

Définition 3.3.2

Analyse de la distribution des propriétés définies pour des noeuds, regroupés selon chaque étiquette attachée à ceux-ci.

3.3.3 - Distribution des propriétés des arcs

Définition 3.3.3

Analyse de la distribution des propriétés définies pour des noeuds, regroupés selon leur ensemble d'étiquettes.

Notons que cette définition restreinte est équivalente à celle de l'analyse par étiquette sous **Neo4j** car les arcs (*Relationships*) ne disposent que d'une seule étiquette.

3.4 - Étiquetage

3.4.1 - Détection d'anomalies par regroupement (clustering)

Définition 3.4.1

On génère à l'aide de l'algorithme **FastRP** un *embedding* à partir des propriétés numériques (les *features*) et de la topologie du graphe pour chaque noeud. Ces *embeddings* sont ensuite

utilisés pour déterminer des groupes (*clusters*) de noeuds avec l'algorithme **KNN**. Une fois ces groupes déterminés on filtre les résultats qui ont une similarité supérieure ou égale à un seuil donné. Enfin on compare les étiquettes (*labels*) des noeuds à celles des autres noeuds pour détecter, le cas échéant, des erreurs d'étiquetage (*labeling*).

Notons que cette méthode est assez fragile, notamment à cause des *embeddings* qui peuvent être en grande partie constitués de valeurs par défaut (*padding*), entraînant un biais conséquent sur les calculs de similarité. D'autres approches comme la détection de communauté avec l'algorithme de **Louvain** seraient envisageables pour cet usage de profilage.

3.5 - Lisibilité

3.5.1 - Distribution du degré des noeuds

Définition 3.5.1

Analyse de la distribution des degrés (entrant et sortant), des noeuds regroupés selon leur ensemble d'étiquettes.

3.5.2 - Détection des arcs formant un multigraphe

Définition 3.5.2

Détection d'arcs partageant le même noeud source et le même noeud destination, formant ainsi un multigraphe. Un tableau de bord concis sur l'ensemble d'étiquettes du noeud source et celui du noeud destination, ainsi que l'ensemble des étiquettes des arcs est construit à partir de cette détection.

3.5.3 - Analyse de l'excentricité du graphe

Définition 3.5.3

Analyse de l'excentricité du graphe : calcul du rayon et du diamètre du graphe. On peut aisément imaginer utiliser ces informations pour analyser un graphe modélisant un réseau par exemple.

3.6 - Valeurs aberrantes (*Outlier*)

3.6.1 - Détection des valeurs numériques aberrantes

Définition 3.6.1

Détection de valeurs numériques aberrantes pour les propriétés des noeuds et des arcs. De nouveau cela permet de caractériser les données et de détecter, le cas échéant, des valeurs invalides.

3.6.2 - Analyse de l'influence transitive des noeuds

Définition 3.6.2

L'influence transitive d'un noeud est déterminée en calculant sa centralité de vecteur propre (*Eigenvector Centrality*); qui est une mesure utilisée en théorie des graphes pour évaluer l'influence d'un noeud. Celle-ci est calculée en tenant compte du nombre de connexions d'un noeud et de l'importance des noeuds auxquels il est connecté.

Cette analyse permet ainsi de mesurer l'influence des noeuds et de détecter, le cas échéant, ceux dont l'influence ne correspond pas au domaine modélisé.

3.6.3 - Analyse de l'influence transitive moyenne

Définition 3.6.3

Analyse de l'influence transitive moyenne à travers les noeuds du graphe.

4 - Implémentation - Neo4j

Neo4j est une base de données graphe proposant une implémentation flexible des graphes de propriété. Les noeuds sont ainsi nommés des « Nodes » et les arcs sont nommés des « Relationships ». L'ensemble des concepts de **Neo4j** est identique à la définition établie en introduction (cf. Définition 1), à l'exception près que les « Relationships » ne peuvent avoir qu'une seule étiquette. Notons que l'implémentation du *framework* de qualité de données établi dans la présente étude a été implémentée cf. [5].

4.1 - Méthodes de test

Dans l'objectif d'évaluer la pertinence des critères de qualité de données que nous avons établis, nous avons testé ceux-ci sur diverses bases de données graphe. Celles-ci comportent des bases de données graphe dont la qualité est irréprochable comme les bases de données utilisées dans la documentation de **Neo4j**. Nous avons besoin de données chaotiques, moins synthétiques, nous avons donc utilisé des bases de données graphe résultant de la transformation de bases de données de connaissances (comme *YAGO3 Knowledge Base*) et dégradé les données.

Nous n'avons pas utilisé de bases de données graphe résultant de la transformation d'une base de données relationnelle afin de s'émanciper du modèle relationnel. De plus nous n'avons pas mené de tests approfondis sur des bases de données **RDF** comme *DBpedia* qui s'est avéré trop pauvre en sémantique, que ce soit par son manque de diversité d'étiquettes ou de propriétés.

4.2 - Résultats des tests

4.2.1 - Northwind

Nous avons utilisé la base de données *Northwind* notamment utilisée dans la documentation de **Neo4j**. Les données de celle-ci présentent de nombreux avantages comme la richesse sémantique – notamment des étiquettes – et nous ont permis de tester certains des indicateurs les plus complexes (comme l'Étiquetage par regroupement - 2.2.5). Nous avons donc dégradé les données pour simuler une base de données dynamique et non synthétique.

Ces dégradations comportent la suppression de 5% des arcs, la corruption de 5% des formats de date, l'invalidation de 5% des contraintes (**FD**, existence, type et unicité) et le changement de l'ensemble d'étiquettes de 2% des noeuds de chaque ensemble distinct d'étiquettes.

Les résultats pour la graine (*seed*) 42 ont été concluants pour la détection d'arcs manquants via l'approche de la définition 2.1.1, de même que pour les indicateurs des définitions : 2.2.2, 2.3.1, 2.3.2, 2.3.3, 2.3.4, 2.4.1. L'Étiquetage par regroupement s'est lui aussi avéré concluant, bien que l'expérimentation démontre certaines faiblesses de son approche.

En effet pour une base de données graphe dont tous les noeuds n'ont qu'une seule étiquette qui leur est associée, le filtrage de la similarité entre ensembles d'étiquettes pour les algorithmes *Merge* et *Split* se polarise en restreignant les valeurs possibles pour la $\text{Similarité}_{\text{Étiquettes}}$ à 0.0 et 1.0. Cela réduit la flexibilité de l'indicateur et peut mener – inévitablement – à des faux positifs (détection d'erreurs qui n'en sont pas). Néanmoins cette approche reste dans la majorité des cas performante pour capturer un sens sémantique complexe et détecter les erreurs d'étiquetage.

La **Figure 19** exhibe une exécution de l'algorithme *Merge* pour laquelle chaque couleur distincte de noeud correspond à un ensemble distinct d'étiquettes. Il est intéressant de voir que la détection d'erreurs d'étiquetage s'illustre par le degré élevé des noeuds. Notons que le graphe présenté est restreint aux arcs e tels que $\lambda(e) = \{\text{Merge}\}$. L'observation plus fine des **Figure 20** et **Figure 21** démontre bien que les noeuds dont l'étiquetage a été dégradé sont

détectés comme fortement similaires à leur véritable étiquetage via la dimension sémantique intacte de leurs relations – les arcs. Bien que notre indicateur construise le graphe en entier et n'exclue aucune relation de similitude; on pourrait aisément imaginer une solution d'analyse basée par exemple sur le degré moyen des noeuds ou encore une méthode de regroupement (*clustering*) pour réduire la quantité de noeuds à analyser.

Enfin la **Figure 22** illustre bien l'erreur d'étiquetage détectée par l'algorithme *Split*. Le noeud central auquel sont liés tous les autres démontre une erreur d'étiquetage manifeste car tous les noeuds qui lui sont liés partagent le même ensemble d'étiquettes. En d'autres termes, les noeuds dont les arcs e tels que $\lambda(e) = \{\text{Split}\}$ sont sortants, sont similaires deux à deux. En toute hypothèse l'indicateur que nous avons défini est donc un outil solide pour analyser la qualité de l'étiquetage d'une base de données graphe.

4.2.2 - YAGO3

La seconde base de données que nous avons utilisée pour tester nos critères de qualité de données est *Yago3* [6]. Afin d'avoir un élément de comparaison significatif nous n'avons effectué qu'une minorité de dégradations sur celle-ci. En effet nous avons dégradé la valeur de 1% des arcs et avons réduit l'échantillon aux 50000 premières lignes (pour lesquelles nous obtenons 63563 noeuds pour 50000 arcs). De plus nous avons utilisé les prédicats des arcs pour étiqueter ceux-ci.

Les résultats des tests conduits ont été particulièrement concluants, ceux-ci ont démontré que nos indicateurs ne détectent aucun problème de qualité de données. Et pour cause : la qualité des données de celle-ci est exemplaire. Les figures **23**, **24** et **25** présentent un infime échantillon (représentatif) des données. L'homogénéité des données est telle que les algorithmes *Merge* et *Split* de l'étiquetage par regroupement n'ont détecté aucune erreur.

Le profilage de la Distribution des propriétés des arcs – 3.3.3, a révélé l'absence d'une propriété importante pour une infime partie des arcs (~0.40%) comme l'illustre la **Figure 26**. Enfin la **Figure 27** illustre l'utilisation de la validation par requête – 2.3.4, afin de détecter les données précédemment dégradées.

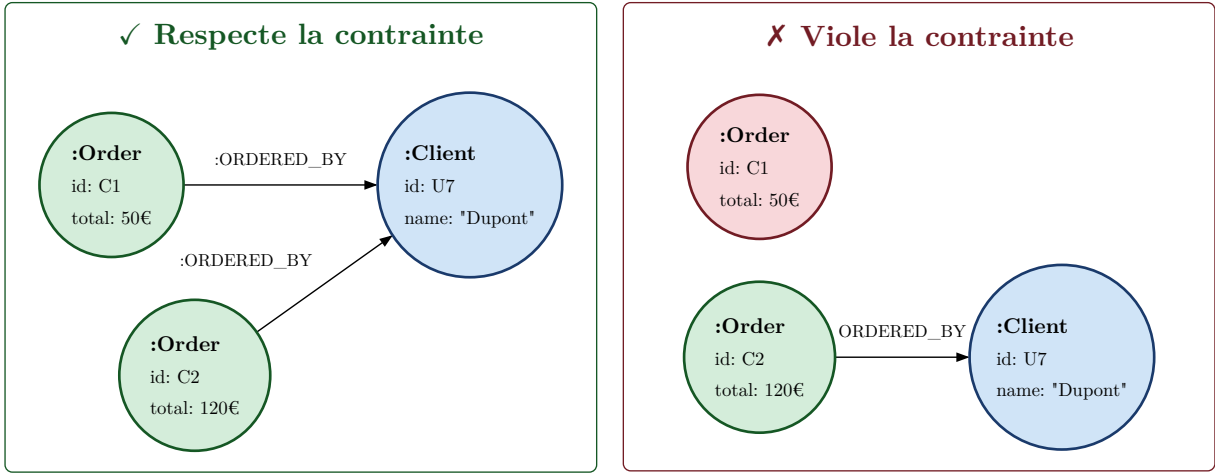
5 - Conclusion

Au terme de cette étude de nombreux indicateurs de qualité de données se sont révélés intéressants et adaptés à un graphe de propriété. De plus lorsque ceux-ci sont couplés avec un système de profilage cela offre une vision d'ensemble sur les données des bases de données graphe. La structure semi-structurée de celles-ci offre un outil puissant pour exprimer des concepts sémantiques complexes. Parvenir à capturer l'ensemble du sens sémantique des bases de données graphe est un enjeu de taille du fait de la pluralité des usages de celles-ci.

Certains défis subsistent donc, que ce soit l'analyse de la qualité de l'étiquetage (*labeling*), la conformité des données (de nombreuses vérifications complexes pourraient être standardisées avec un *DDL*) ou encore les formes normales d'une base de données graphe.

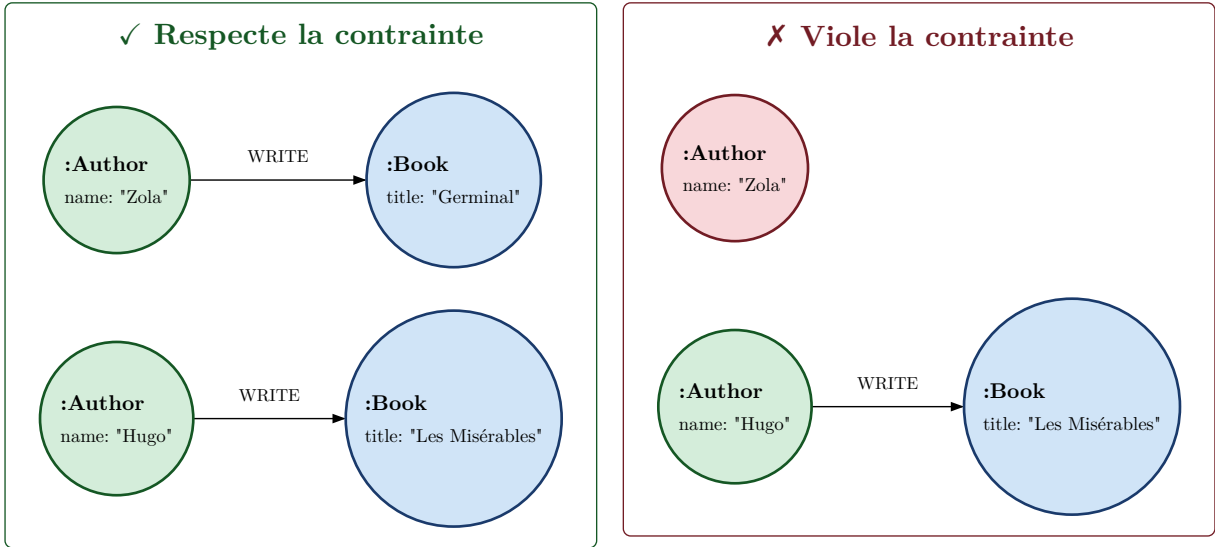
6 - Annexe

Cette annexe rassemble des figures de graphes illustrant les définitions précédemment établies.



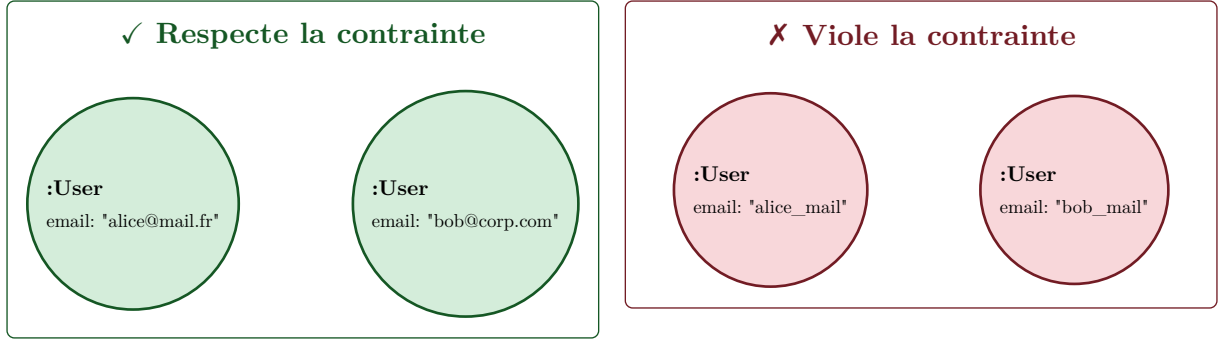
$$(G_p, O, L_O) = ((\{ : \text{Order} \}) - [\{ : \text{ORDERED_BY} : 1 \}] \rightarrow (\{ : \text{Client} \}), \{ N \}, \{ \text{Order} \})$$

Figure 1 : Exemple pour la *Définition 2.1.1*



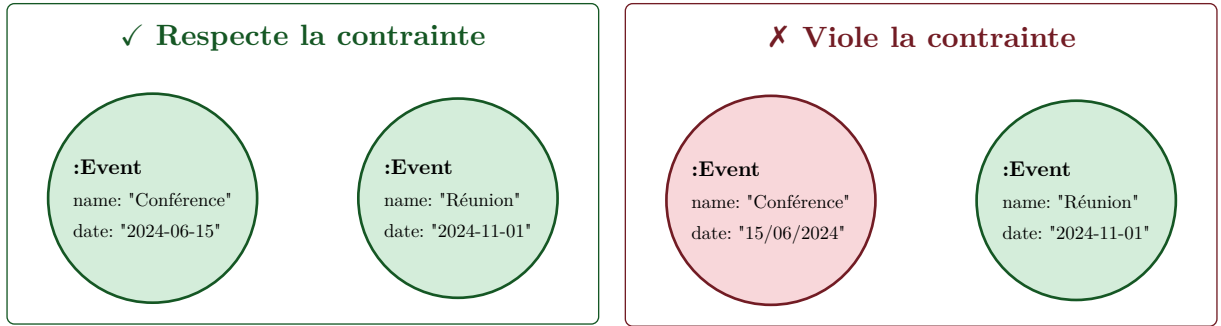
$$(L_O, D_s, D_e) = (\{ \text{Author} \}, \{ 1, 2, 3 \}, \{ 0 \})$$

Figure 2 : Exemple pour la *Définition 2.1.2*



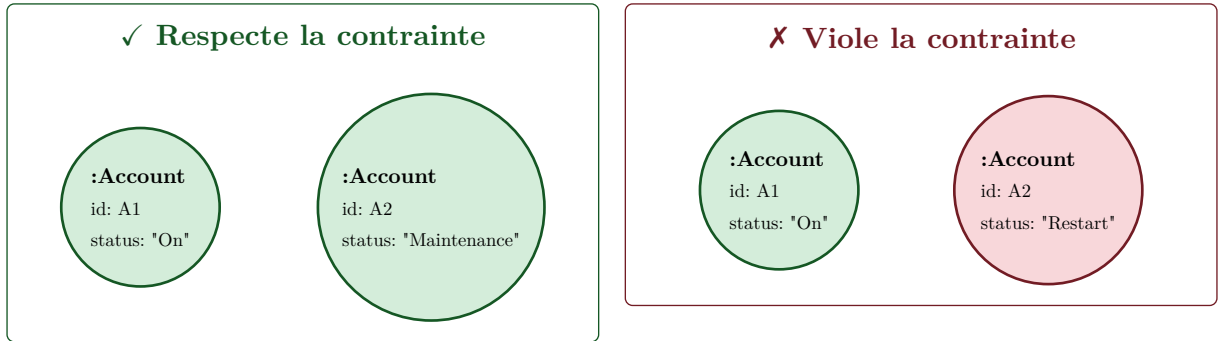
$$(O, L_O, X, \text{Regex}) = (N, \{\text{User}\}, \{\text{email}\}, "/^[\w.]+\@[\w.]+\.[a-z]\{2,\}")$$

Figure 3 : Exemple pour la Définition 2.2.1



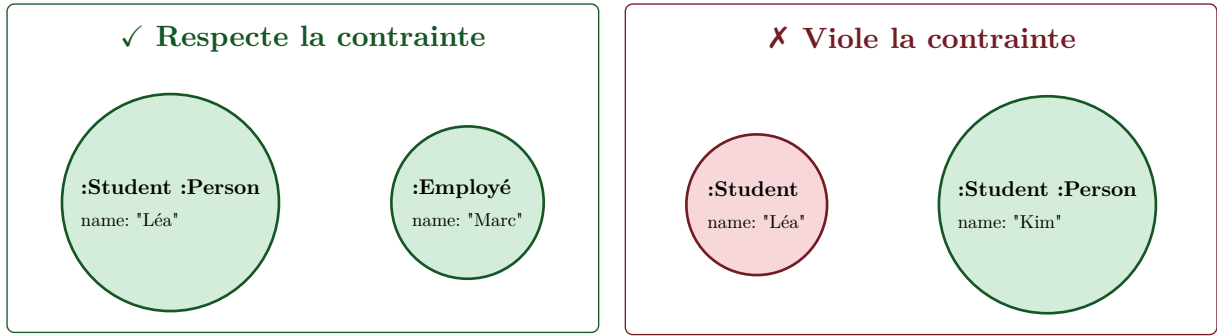
$$(O, L_O, X, \text{Date}_{\text{fmt}}) = (N, \{\text{Event}\}, \{\text{date}\}, \text{"YYYY-MM-DD"})$$

Figure 4 : Exemple pour la Définition 2.2.2



$$(O, L_O, X, I) = (N, \{\text{Account}\}, \{\text{status}\}, \{\text{On}, \text{Off}, \text{Maintenance}\})$$

Figure 5 : Exemple pour la Définition 2.2.3



$(O, L_X, L_Y, Op_{\text{ens}}) = (N, \{\text{Student}\}, \{\text{Person}\}, \subset)$
Figure 6 : Exemple pour la Définition 2.2.4

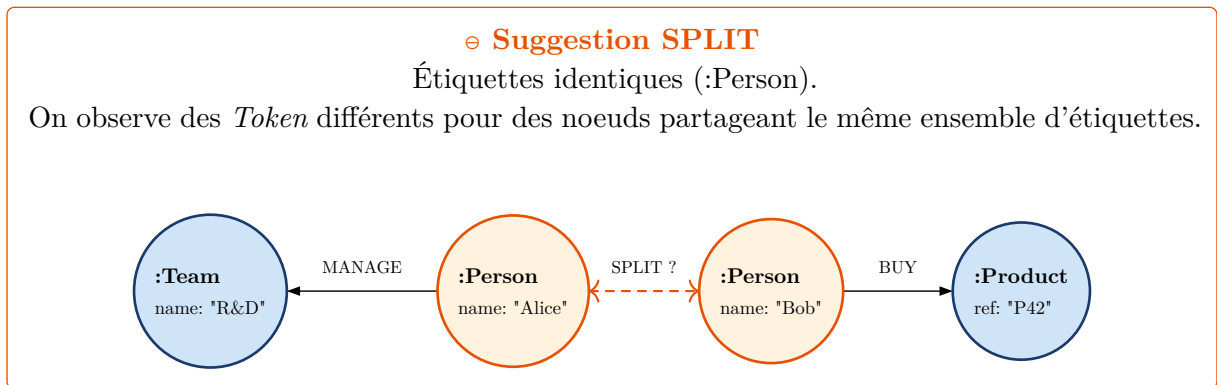
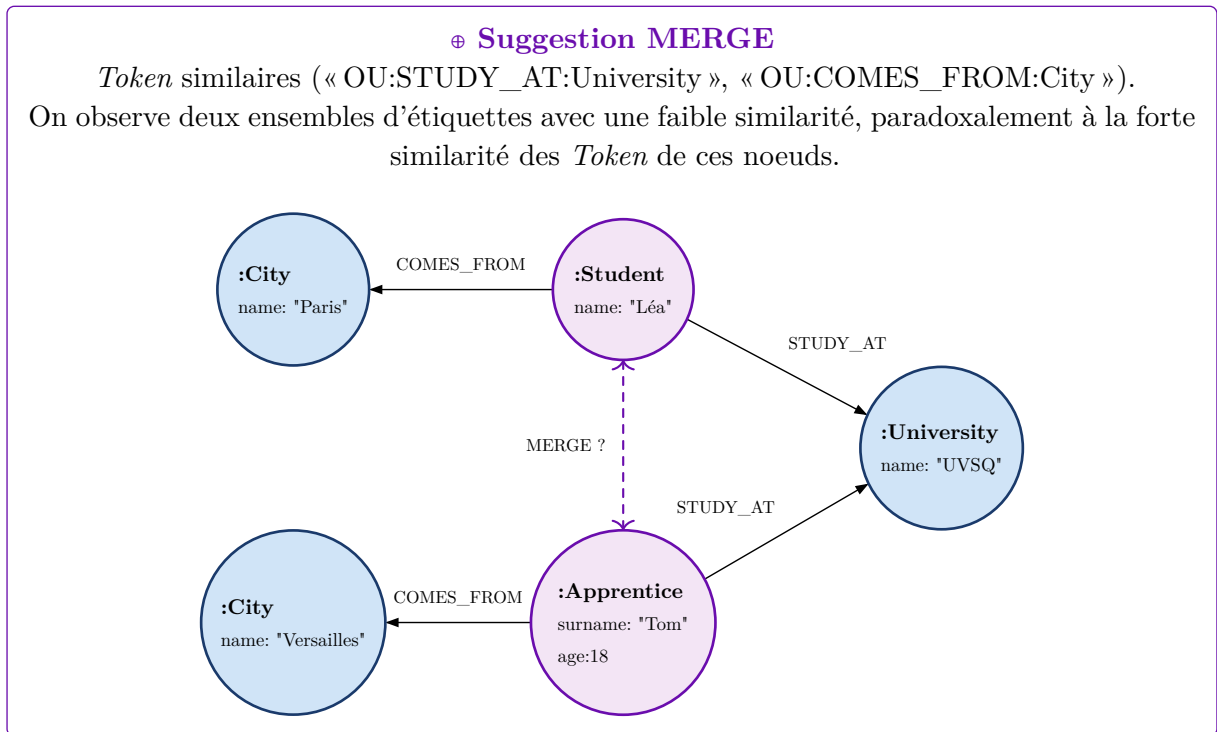
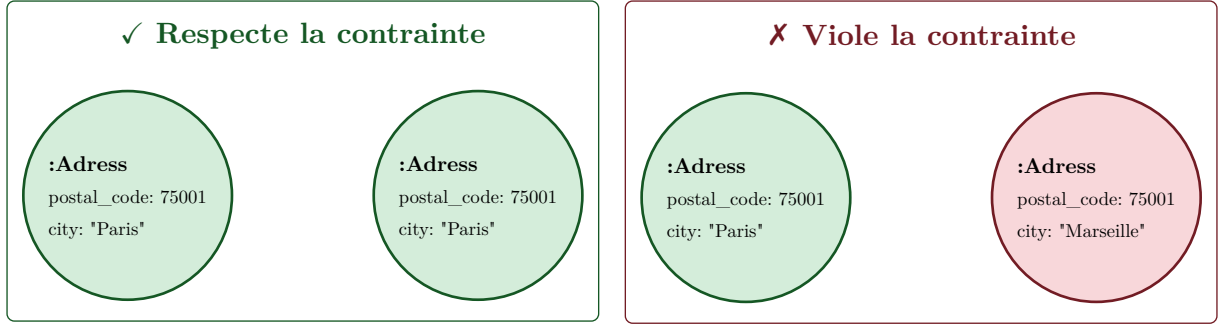
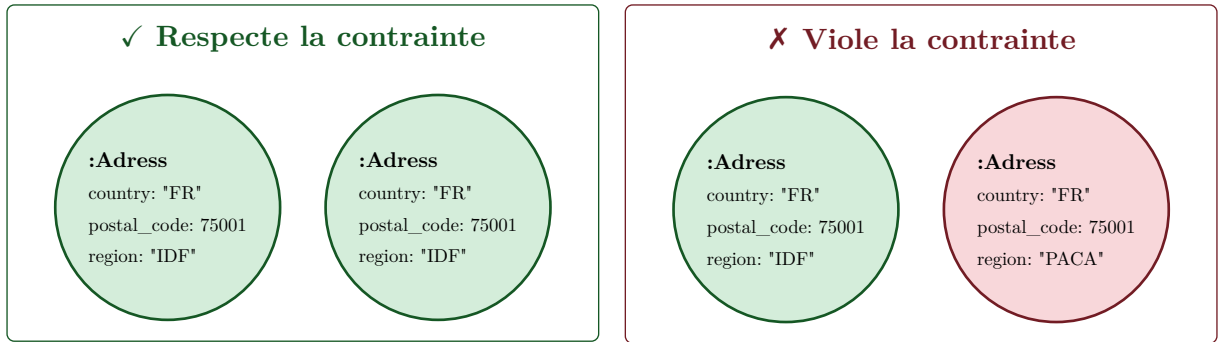


Figure 7 : Exemple pour la Définition 2.2.5



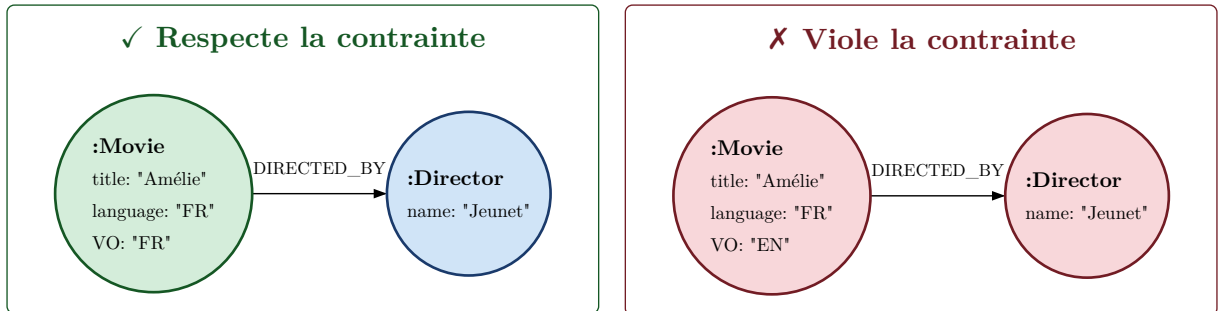
$$(O, L_O, X \rightarrow Y) = (N, \{\text{Adress}\}, \{\text{postal_code}\} \rightarrow \{\text{city}\})$$

Figure 8 : Exemple pour la *Définition 2.3.1*



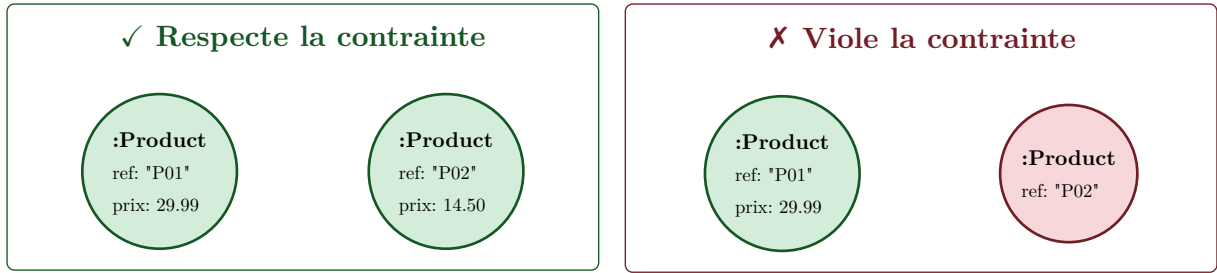
$$(O, L_O, C, X \rightarrow Y) = (N, \{\text{Adress}\}, (\{\text{country}\}, \text{"FR"}, =, \emptyset), \{\text{postal_code}\} \rightarrow \{\text{region}\})$$

Figure 9 : Exemple pour la *Définition 2.3.2*



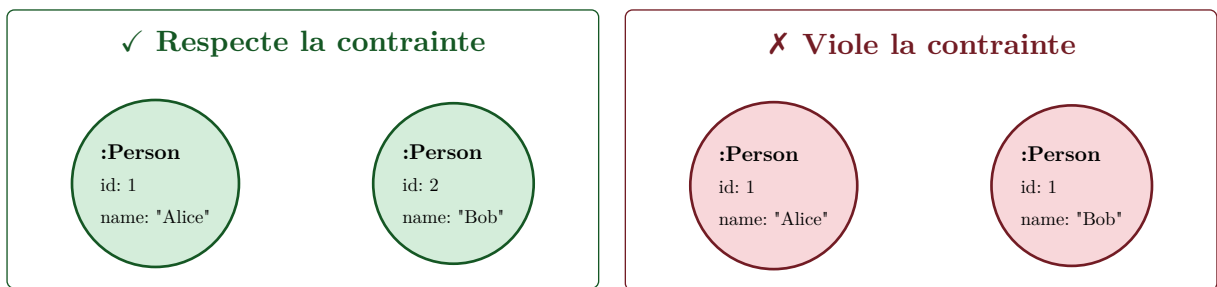
$$(G_p, O, L_O, X \rightarrow Y) = ((\{ : \text{Movie} \}) - [\{ : \text{DIRECTED_BY} \} : 1] \rightarrow (\{ : \text{Director} \}), \{ N \}, \{ \text{language} \} \rightarrow \{ \text{VO} \})$$

Figure 10 : Exemple pour la *Définition 2.3.3*



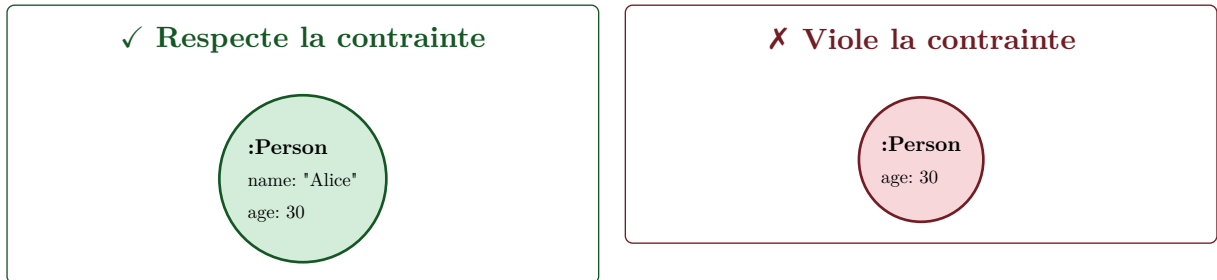
$(R, B) = (\text{"MATCH (n:Product) WHERE n.prix IS NULL"}, \text{False})$

Figure 11 : Exemple pour la *Définition 2.3.4*



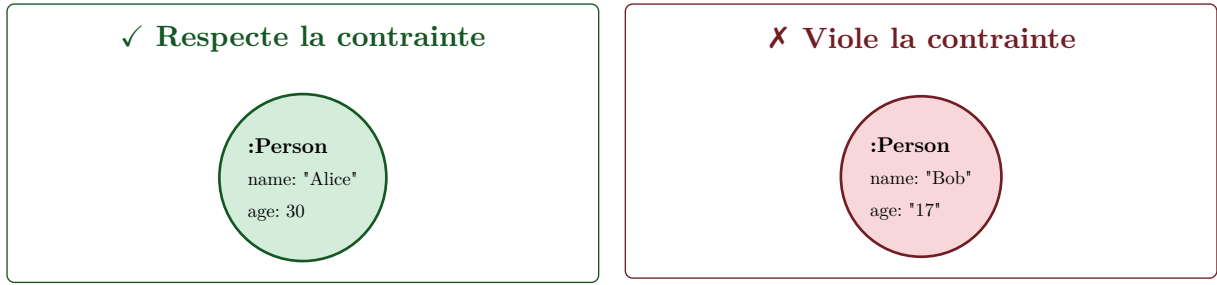
$(O, L_O, X) = (N, \{\text{Person}\}, \{\text{id}\})$

Figure 12 : Exemple pour la *Définition 1 de 2.4.1*



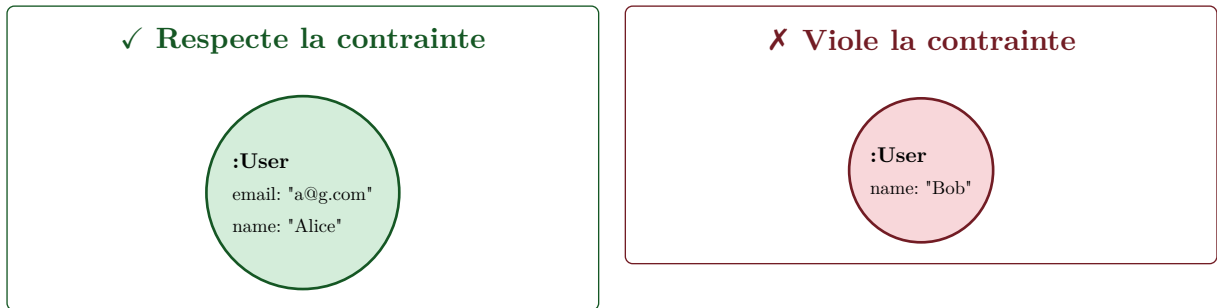
$(O, L_O, X) = (N, \{\text{Person}\}, \{\text{name}\})$

Figure 13 : Exemple pour la *Définition 2 de 2.4.1*



$$(O, L_O, X, Y) = (N, \{\text{Person}\}, \{\text{age}\}, \{\text{Integer}\})$$

Figure 14 : Exemple pour la *Définition 3* de 2.4.1



$$(O, L_O, \text{Index}, X) = (N, \{\text{User}\}, \{\text{email}\}, \{\text{email}\})$$

Figure 15 : Exemple pour la *Définition 2.4.2*

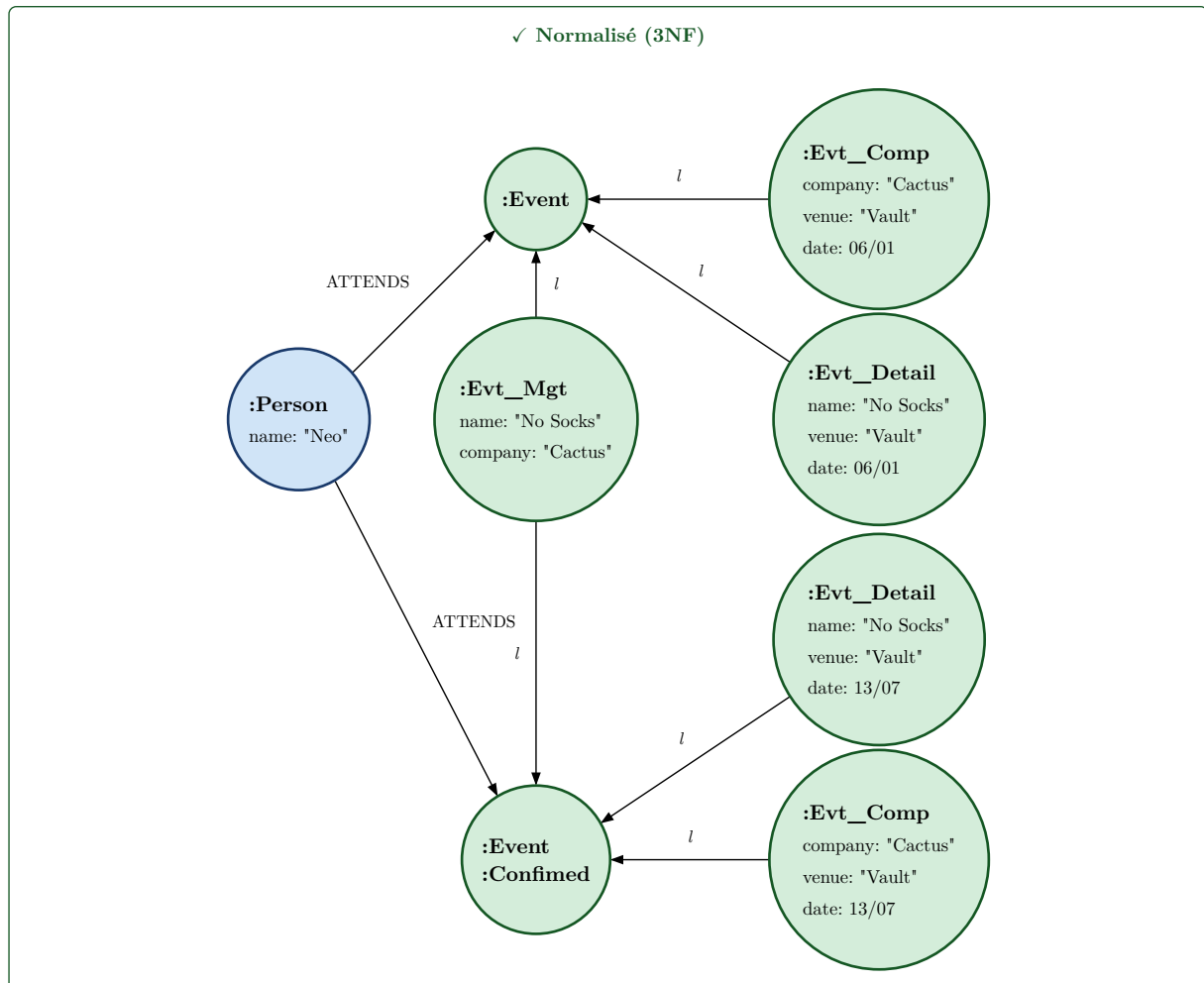
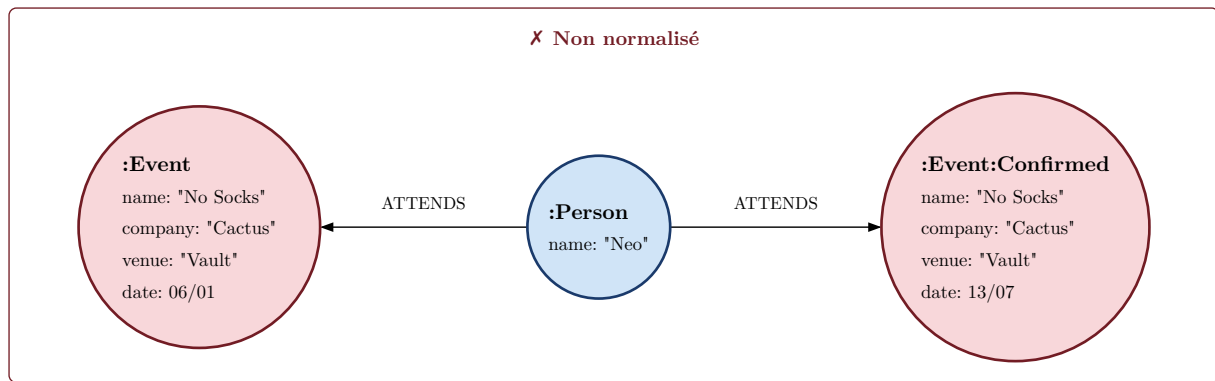


Figure 16 : Exemple pour la *Définition 2.4.3*

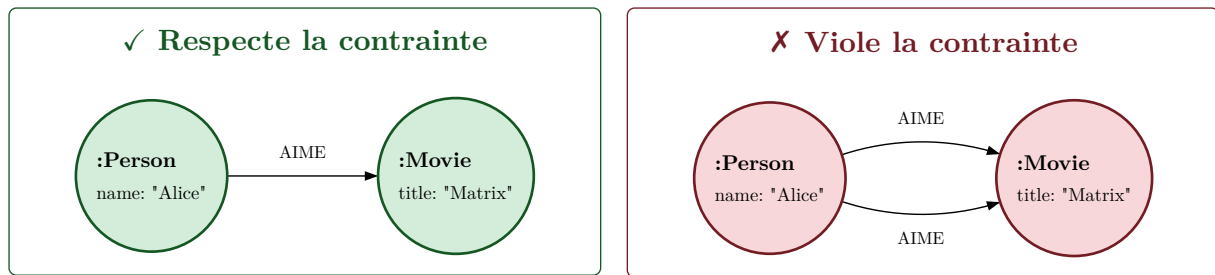


Figure 17 : Exemple pour la *Définition 2.5.1*

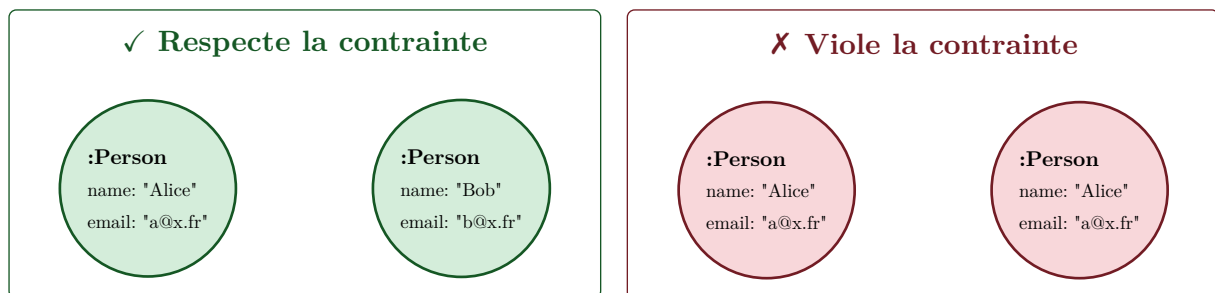
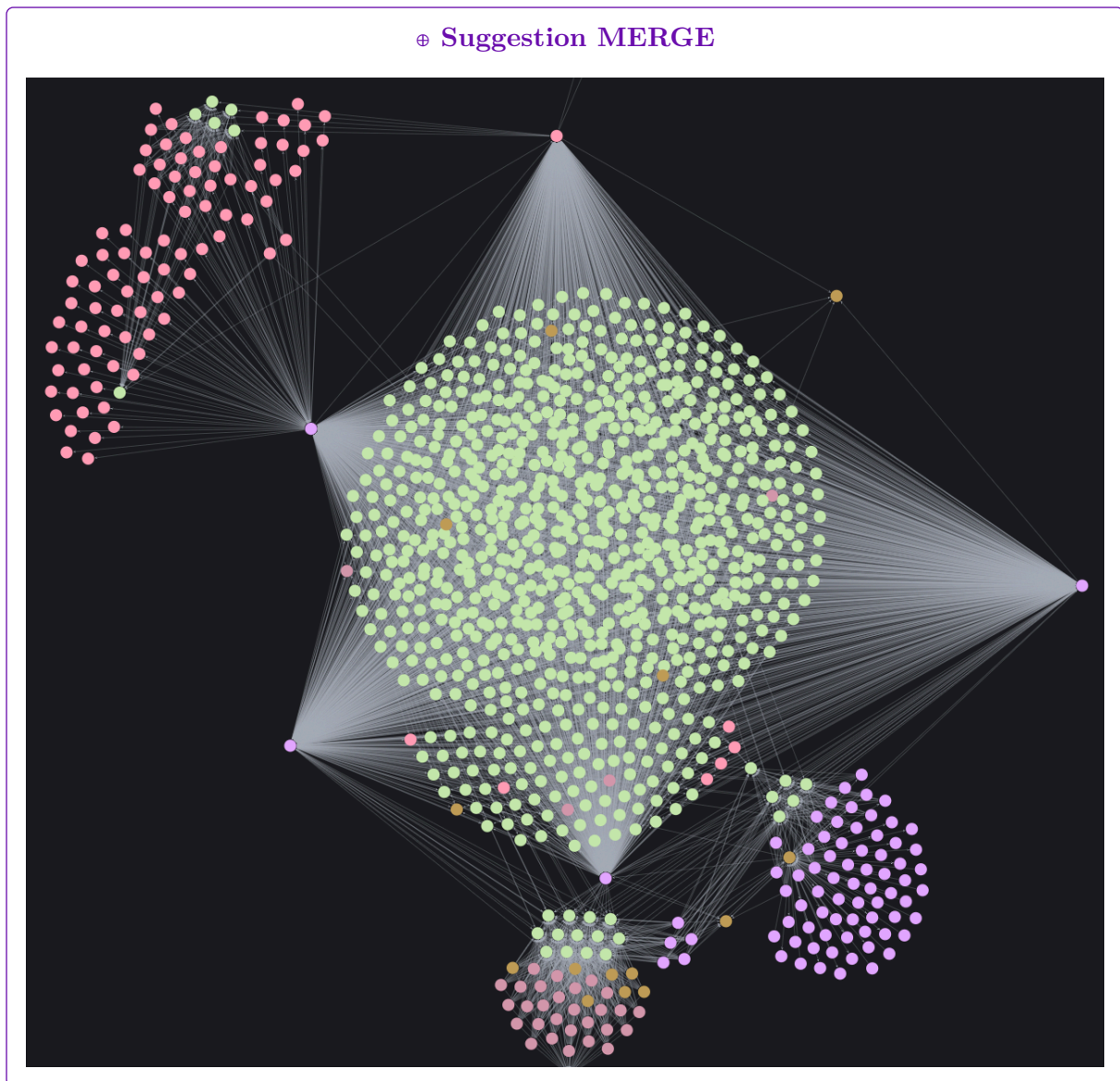


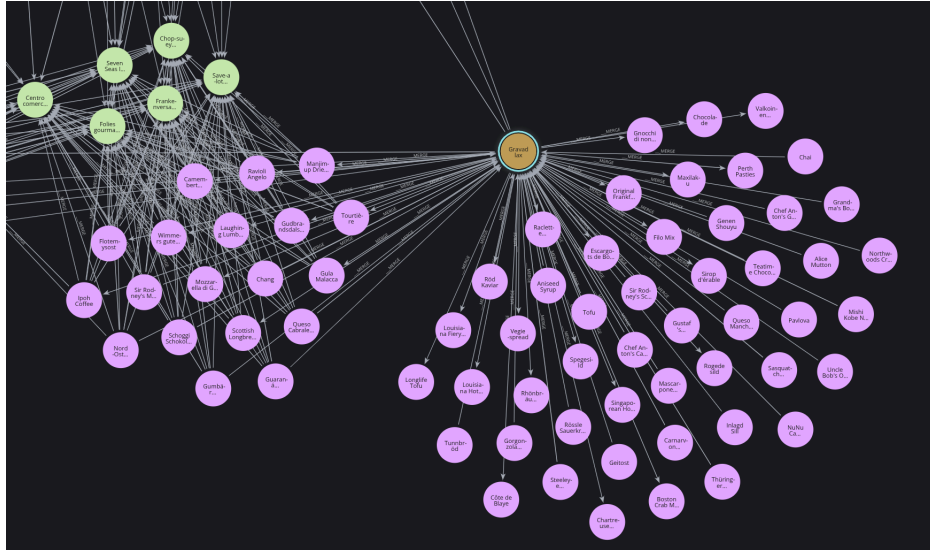
Figure 18 : Exemple pour la *Définition 2.5.2*



Base de données *Northwind* (cf. 4.2.1) dégradée avec la *seed* 42 et
Merge exécuté avec les arguments $(t_e, t_t) = (0.4, 0.6)$.

Figure 19 : Exécution de l'algorithme *Merge* sur la base de données *Northwind*

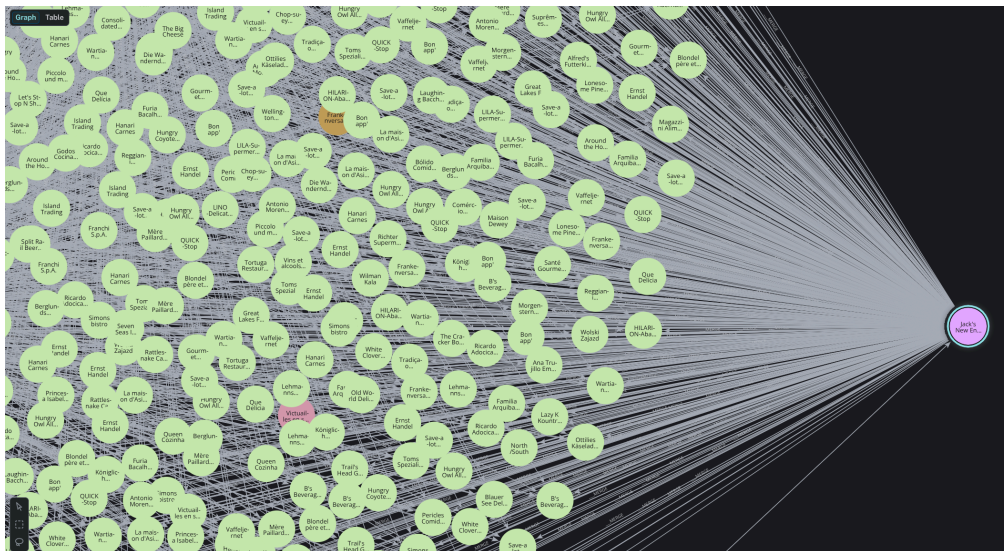
⊕ Suggestion MERGE



Base de données *Northwind* (cf. 4.2.1) dégradée avec la *seed* 42 et
Merge exécuté avec les arguments $(t_e, t_t) = (0.4, 0.6)$.

Figure 20 : Exécution de l'algorithme *Merge* sur la base de données *Northwind*

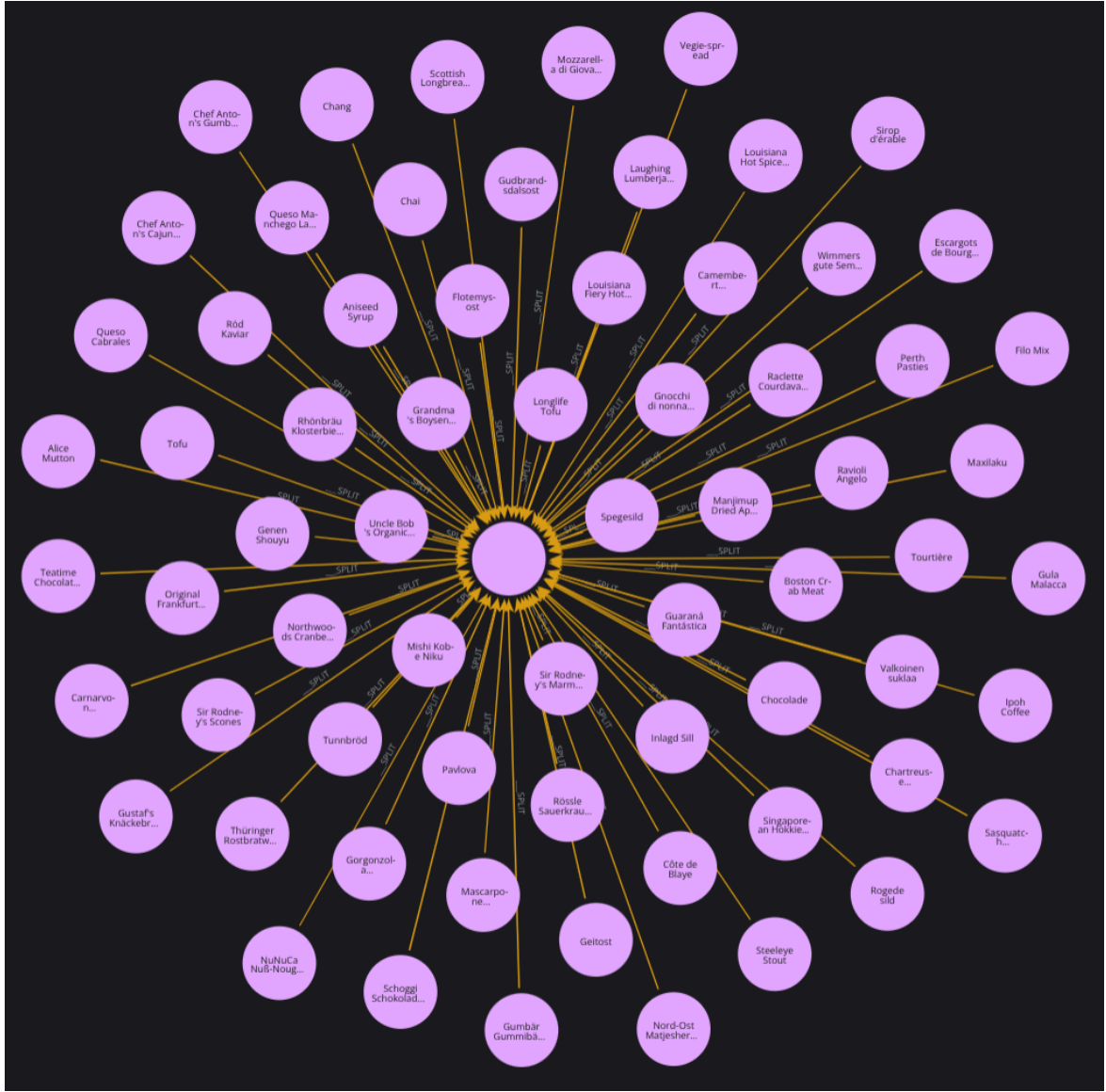
⊕ Suggestion MERGE



Base de données *Northwind* (cf. 4.2.1) dégradée avec la *seed* 42 et
Merge exécuté avec les arguments $(t_e, t_t) = (0.4, 0.6)$.

Figure 21 : Exécution de l'algorithme *Merge* sur la base de données *Northwind*

⊖ Suggestion SPLIT



Base de données *Northwind* (cf. 4.2.1) dégradée avec la *seed* 42 et
Split exécuté avec les arguments $(t_e, t_t) = (0.7, 0.4)$.

Figure 22 : Exécution de l'algorithme *Split* sur la base de données *Northwind*

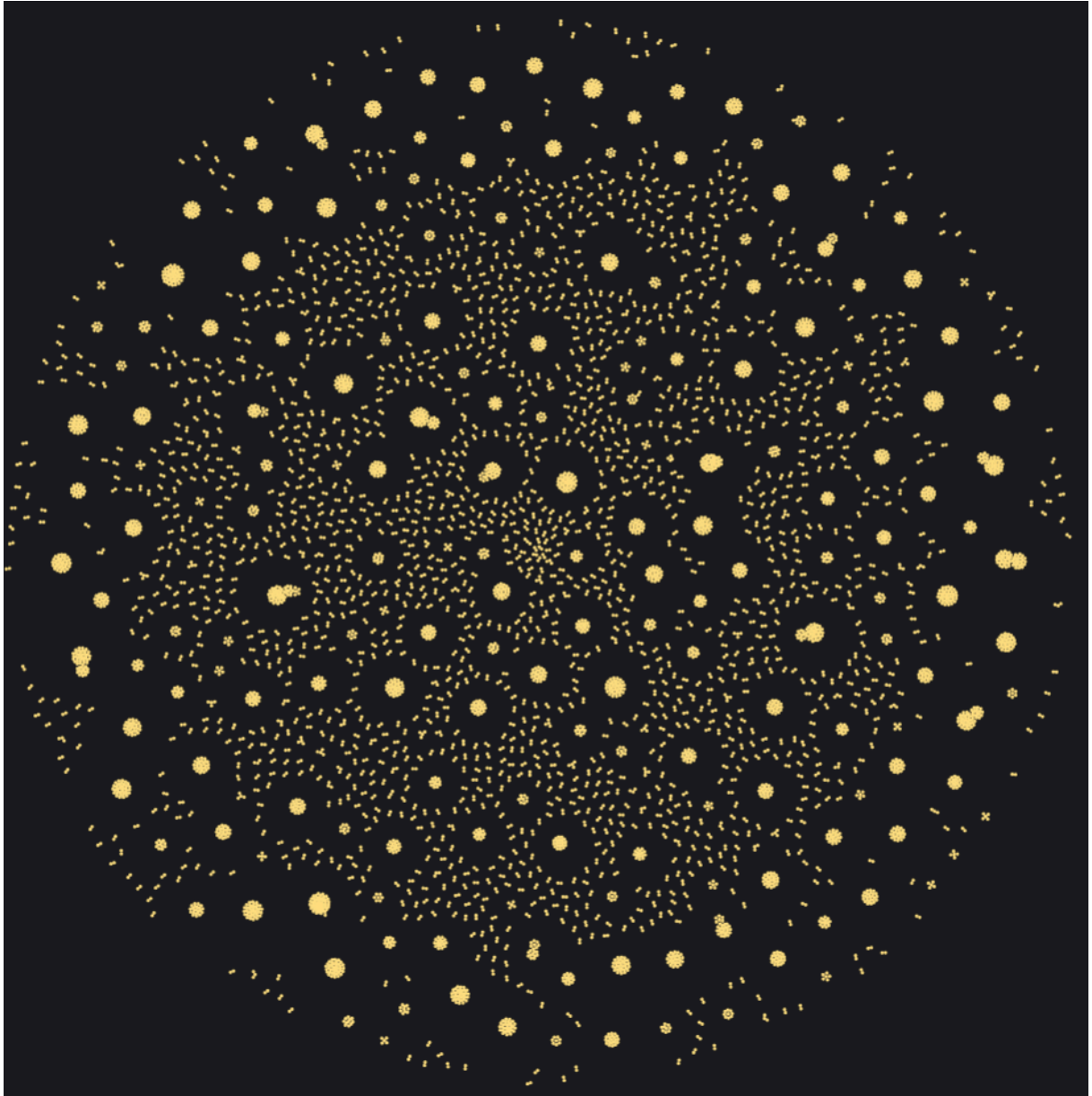
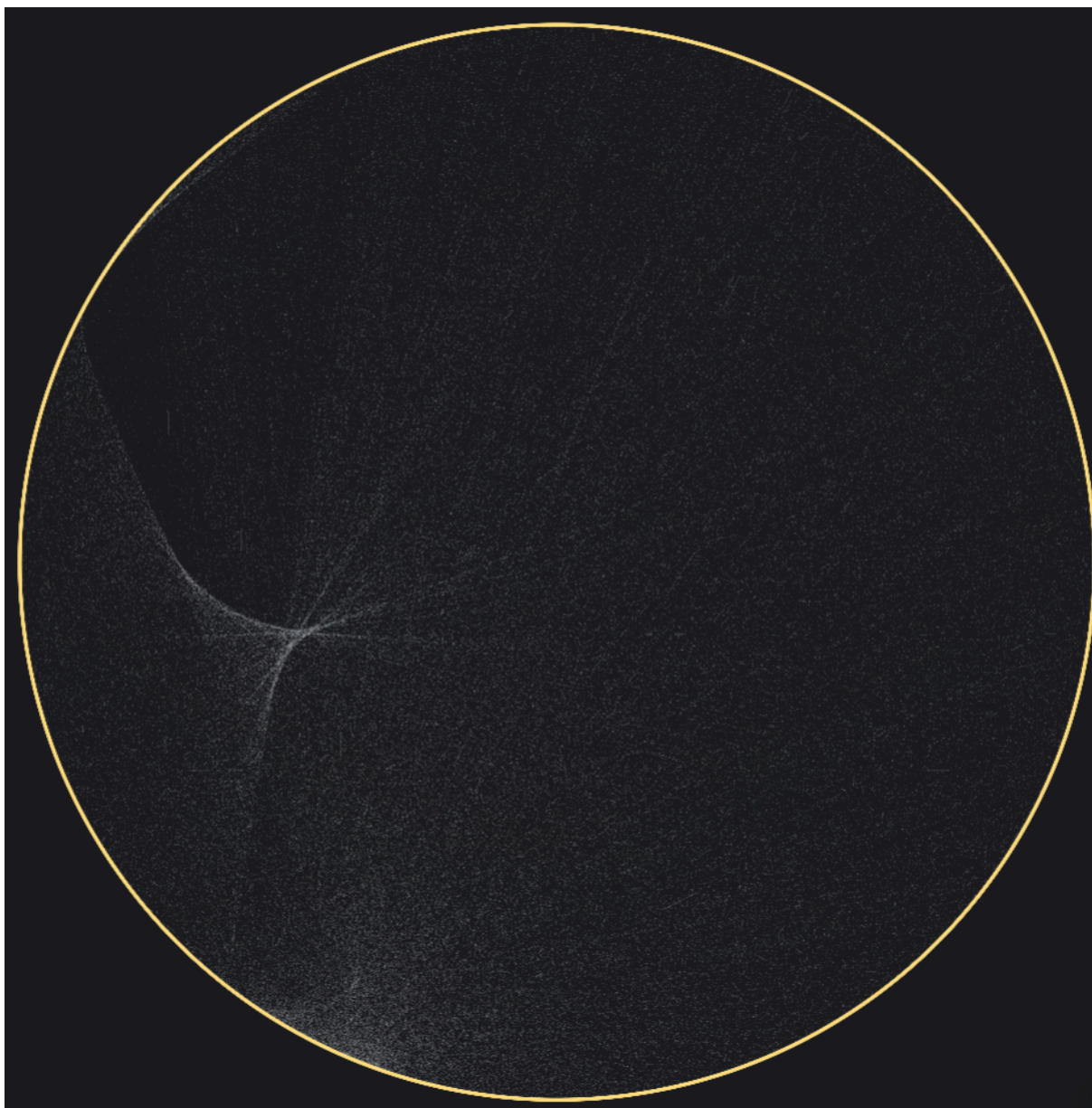


Figure 23 : Échantillon de la base de données *YAGO3* – 4.2.2



Les segments blanc modélisent les arcs et les noeuds forment le cercle.
Figure 24 : Échantillon de la base de données *YAGO3* – 4.2.2

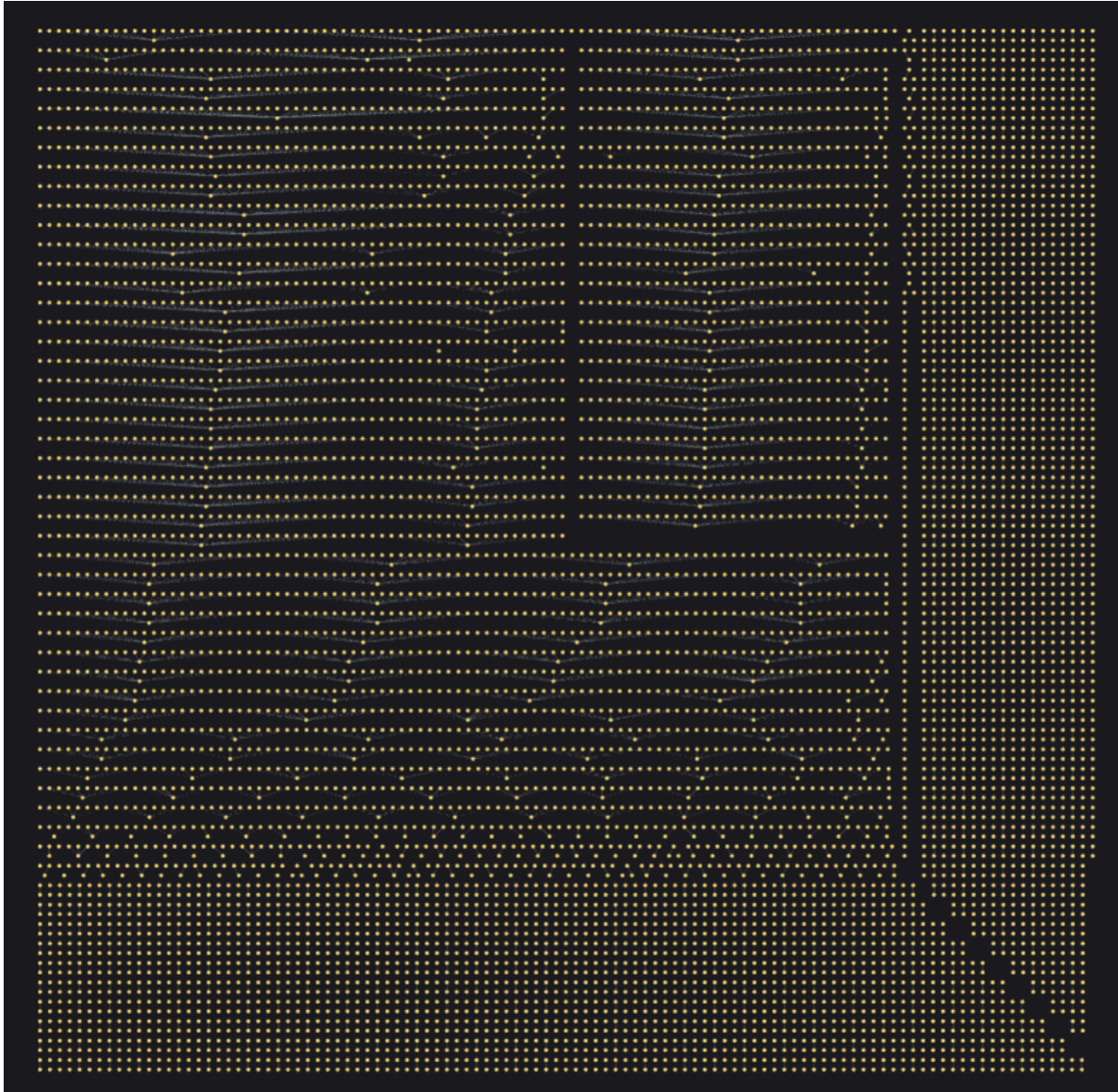


Figure 25 : Échantillon de la base de données *YAGO3* – 4.2.2

	entity	label	count	properties_names	properties_count	properties_percent
0	Entity.RELATIONSHIP	<hasGloss>	1		1	100
1	Entity.RELATIONSHIP	<wasCreatedOnDate>	16349	date_value	16349	100
2	Entity.RELATIONSHIP	<diedOnDate>	10052	date_value	10046	99.9403
3	Entity.RELATIONSHIP	<diedOnDate>	10052		6	0.0597
4	Entity.RELATIONSHIP	<wasBornOnDate>	20850	date_value	20847	99.9856
5	Entity.RELATIONSHIP	<wasBornOnDate>	20850		3	0.0144
6	Entity.RELATIONSHIP	<wasDestroyedOnDate>	1231	date_value	1231	100
7	Entity.RELATIONSHIP	<happenedOnDate>	1517	date_value	1512	99.6704
8	Entity.RELATIONSHIP	<happenedOnDate>	1517		5	0.3296

Illustration de l'interface utilisateur de *data-quality* [5].

Figure 26 : Profilage des propriétés des arcs de la base de données *YAGO3* – 4.2.2

CDVQ - Data validation query

Validate your data with a query approach for complex purpose.

	Query	Should be empty
	match ()-[r]-() where r.date_value = "3000-01-01" return r	☒

Analyse

Result of the analysis:

	query	should_be_empty	found
0	match ()-[r]-() where r.date_value = "3000-01-01" return r	☒	926

Illustration de l'interface utilisateur de **data-quality** [5].

Figure 27 : Validation par requête 2.3.4 – de la base de données YAGO3 – 4.2.2

Bibliographie

- [1] L. Cai et Y. Zhu, « The Challenges of Data Quality and Data Quality Assessment in the Big Data Era », *Data Science Journal*, vol. 14, p. Art.2, 1-10, 2016, doi: 10.5334/dsj-2015-002.
- [2] R. Giot, R. Bourqui, N. Journet, et A. Vialard, « Visual Graph Analysis for Quality Assessment of Manually Labelled Documents Image Database », in *13th International Conference on Document Analysis and Recognition (ICDAR)*, Tunis, Tunisia, 2015. [En ligne]. Disponible sur: <https://hal.science/hal-01170011>
- [3] M. Manouvrier et K. Belhajjame, « PG-FD: Mapping Functional Dependencies to the Future Property Graph Schema Standard », in *Proceedings of the 24th European Conference on Advances in Databases and Information Systems (ADBIS)*, 2024, p. 45-59. doi: 10.1007/978-3-031-70626-4_4.
- [4] P. Skavantzios et S. Link, « Normalizing Property Graphs », *Proceedings of the VLDB Endowment (PVLDB)*, vol. 16, n° 11, p. 3031-3043, 2023, doi: 10.14778/3611479.3611506.
- [5] L. Desmarès, « data-quality ». [En ligne]. Disponible sur: <https://github.com/LugolBis/data-quality>
- [6] F. Mahdisoltani, J. Biega, et F. Suchanek, « Yago3: A knowledge base from multilingual wikipedias », in *7th biennial conference on innovative data systems research*, 2014. [En ligne]. Disponible sur: <https://yago-knowledge.org/downloads/yago-3>